FUNDACIÓN UNIVERSITARIA AGRARIA DE COLOMBIA- UNIAGRARIA

ACTIVIDAD 2: CONFIGURACIÓN DE HERRAMIENTAS

TUTORIAL DE GIT Y GITHUB

CAPITULO 1- INTRODUCCIÓN A GIT Y GITHUB

Presentado por

Laura Camila Monroy Díaz

Asignatura

Ciencia de datos

Docente

Carlos Andres Gómez Vasco

Bogotá D.C

2026

1.1 ¿QUÉ ES GIT?

Git es un sistema de control de versiones que permite registrar los cambios que se realizan en los archivos de un proyecto. En otras palabras, es una herramienta que ayuda a llevar un seguimiento del trabajo desde el momento en que se empieza hasta las diferentes modificaciones que se hacen durante su desarrollo.

Para entenderlo mejor, imaginemos que estamos realizando un trabajo universitario en el que debemos escribir información, corregir errores y agregar nuevos temas. Si guardamos el documento varias veces, podemos terminar con muchos archivos diferentes, como «trabajo final», «trabajo final corregido» y «trabajo final definitivo». Esto puede generar confusión, porque no siempre sabemos cuál es la última versión o qué cambios se hicieron. ayuda a solucionar este problema porque permite registrar los cambios de manera organizada y conservar un historial del proyecto. De esta forma, podemos revisar qué se modificó, cuándo se guardaron los cambios y consultar versiones anteriores cuando sea necesario.

Una característica importante de Git es que funciona principalmente en el equipo donde se está trabajando. Por eso, es posible utilizarlo para llevar el control de un proyecto sin necesidad de tener conexión permanente a internet. Sin embargo, cuando queremos compartir nuestro trabajo con otras personas o almacenarlo en una plataforma en línea, podemos utilizar Git junto con servicios como GitHub.

En conclusión, Git es una herramienta que facilita la organización de los archivos, permite llevar un registro de los avances y ayuda a mantener un mejor control de los proyectos.

1.2. ¿PARA QUÉ SIRVE GIT?

Git sirve para administrar los cambios que se realizan en un proyecto. Aunque es muy utilizado por programadores para desarrollar aplicaciones y páginas web, también puede ser útil para organizar documentos, archivos de texto y otros materiales digitales.

Una de sus principales ventajas es que permite guardar los avances de un proyecto sin tener que crear manualmente una copia del archivo cada vez que se hace una modificación.

Entre sus principales funciones se encuentran las siguientes:

1. **Registrar los cambios realizados**

Git permite guardar un registro de las modificaciones que se hacen en los archivos. Por ejemplo, si estamos escribiendo un documento y agregamos un nuevo apartado, podemos registrar ese avance para que quede incluido en el historial del proyecto.

2. **Consultar versiones anteriores**

En ocasiones, después de realizar una modificación, podemos darnos cuenta de que la versión anterior era mejor o contenía información que necesitamos recuperar. Git permite consultar versiones anteriores y, dependiendo de la situación, recuperar información sin tener que empezar todo el trabajo desde cero.

3. **Mantener organizado el proyecto**

Git permite tener un historial de los cambios realizados. Esto ayuda a comprender cómo ha evolucionado un proyecto y facilita la identificación de las modificaciones que se han hecho en cada etapa.

4. **Facilitar el trabajo en equipo**

Cuando varias personas participan en un mismo proyecto, es importante tener una forma organizada de controlar los cambios. Git permite que cada integrante trabaje en diferentes partes y ayuda a integrar las modificaciones realizadas, aunque en algunos casos es necesario resolver conflictos cuando varias personas modifican las mismas líneas de un archivo.

5. **Identificar los cambios**

Git permite revisar qué partes de los archivos se han modificado. Esto es útil para comprobar el trabajo antes de guardarlo definitivamente en el historial y evitar registrar cambios que todavía necesitan correcciones.

Por ejemplo, en esta actividad académica, Git puede utilizarse para llevar el control de los documentos relacionados con el aprendizaje de esta herramienta. Si se corrige una explicación o se agrega información sobre un comando, es posible registrar esa modificación y mantener un historial organizado.

1.3. ¿QUÉ ES EL CONTROL DE VERSIONES?

El control de versiones es un sistema que permite registrar y administrar los cambios que se realizan en uno o varios archivos a lo largo del tiempo.

Su propósito es conservar un historial organizado del trabajo para que sea posible consultar las modificaciones anteriores, identificar quién realizó determinados cambios y comprender cómo ha evolucionado un proyecto.

Existen diferentes herramientas para llevar el control de versiones, pero Git es una de las más conocidas debido a que es gratuita, de código abierto y puede utilizarse en distintos sistemas operativos.

Para comprender mejor este concepto, pensemos en un trabajo universitario que comienza con una versión sencilla. Después, se agregan nuevos temas, se corrigen errores y se reorganiza la información. Con un sistema de control de versiones, podemos registrar estos avances en diferentes momentos y consultar cómo estaba el proyecto antes de cada modificación. En Git, estos registros se conocen como commits. Cada commit representa un conjunto de cambios guardados en el historial y normalmente incluye un mensaje que explica qué se modificó.

Por ejemplo, un mensaje de commit podría ser: «Agregar introducción al documento». Este mensaje permite identificar fácilmente el propósito de los cambios registrados.

Es importante aclarar que Git no guarda automáticamente cada cambio que hacemos en un documento. Primero debemos preparar los archivos que queremos registrar y después crear un commit para guardar esos cambios en el historial. Por esta razón, aprender a utilizar Git implica conocer cómo revisar los archivos modificados, preparar los cambios y registrarlos correctamente.

1.4. ¿QUÉ ES GITHUB?

GitHub es una plataforma en línea que permite alojar repositorios y compartir proyectos con otras personas. Se utiliza frecuentemente junto con Git porque facilita el almacenamiento de los proyectos en internet y el trabajo colaborativo.

Mientras Git se encarga de llevar el control de versiones, GitHub ofrece un espacio donde podemos publicar nuestros repositorios, consultar sus archivos y permitir que otras personas revisen el contenido, según los permisos establecidos.

Por ejemplo, un estudiante puede crear un repositorio en GitHub para guardar los documentos de una actividad universitaria. Allí puede incluir archivos de texto, documentos de apoyo, instrucciones y un archivo README que explique el contenido del proyecto. Además, GitHub permite visualizar el historial de cambios que se hayan enviado a la plataforma, consultar los mensajes de los commits y revisar las modificaciones registradas.

Otra ventaja es que los repositorios pueden configurarse como públicos o privados. Un repositorio público puede ser consultado por otras personas, mientras que uno privado limita el acceso a las personas autorizadas. Esto resulta útil en actividades académicas porque el estudiante puede compartir el enlace de su repositorio con el docente para que revise los archivos y las evidencias del trabajo realizado.

Sin embargo, es importante tener presente que crear o modificar archivos directamente desde la página web de GitHub no demuestra, por sí solo, que se hayan ejecutado los comandos de Git desde una terminal. Si una actividad exige practicar comandos como git add, git commit y git push, es necesario realizar y documentar ese proceso de acuerdo con las instrucciones del docente.

1.5. DIFERENCIAS ENTRE GIT Y GITHUB

Aunque Git y GitHub están relacionados, cumplen funciones diferentes. Por eso, es importante aprender a distinguirlos.

Git es una herramienta de control de versiones. Permite registrar los cambios de los archivos, consultar el historial y administrar las versiones de un proyecto desde un repositorio local.

GitHub es una plataforma en línea que permite alojar repositorios Git y compartirlos con otras personas. También ofrece funciones para colaborar, revisar cambios y administrar proyectos.

Una forma sencilla de entender la diferencia es imaginar que estamos realizando un trabajo universitario. Git sería la herramienta que nos ayuda a registrar las modificaciones que hacemos en los documentos. GitHub sería el espacio en internet donde podemos alojar una copia del repositorio para consultarlo y compartirlo.

Las principales diferencias son las siguientes:

- **Funcionamiento:** Git puede utilizarse localmente en un equipo, mientras que GitHub es una plataforma en línea.
- **Propósito:** Git controla las versiones de los archivos y GitHub permite alojar y compartir repositorios.
- **Conexión a internet:** muchas operaciones locales de Git pueden realizarse sin conexión, pero para enviar cambios a GitHub normalmente se necesita internet.
- **Colaboración:** Git permite gestionar cambios y combinar versiones; GitHub ofrece herramientas adicionales para que las personas colaboren en los proyectos.
- **Historial:** Git registra los cambios en el repositorio y GitHub permite consultar los cambios que se han enviado a la plataforma.

En conclusión, Git y GitHub se complementan. Git permite controlar las versiones del proyecto y GitHub facilita su almacenamiento en línea y su colaboración con otras personas.

1.6. ¿QUÉ ES UN REPOSITORIO?

Un repositorio es el espacio donde se almacenan los archivos de un proyecto y donde Git lleva el registro de sus cambios.

En un repositorio podemos encontrar documentos, imágenes, archivos de texto, código yotros elementos necesarios para desarrollar una actividad. Además, cuando utilizamos Git, el repositorio conserva el historial de los cambios que se han registrado mediante commits.

Existen dos conceptos importantes: el repositorio local y el repositorio remoto.

- **Repositorio local:** es el repositorio que se encuentra en nuestro equipo. Desde allí podemos modificar archivos, revisar los cambios y registrar avances utilizando los comandos de Git.

- **Repositorio remoto:** es una versión del repositorio alojada en otro lugar, por ejemplo, en GitHub. Permite compartir el proyecto y sincronizar los cambios entre diferentes ubicaciones.

Por ejemplo, si creamos un repositorio para una actividad académica, podemos tener una copia local para trabajar y otra en GitHub para compartirla con el profesor.

Cuando realizamos cambios en el repositorio local, podemos enviarlos al repositorio remoto mediante git push. De esta manera, los cambios quedan disponibles en GitHub. Un repositorio bien organizado facilita la revisión del proyecto, porque permite identificar los archivos, consultar su contenido y conocer los cambios que se han registrado.

1.7. ¿QUÉ ES GIT BASH O LA TERMINAL?

Una terminal es una herramienta que permite comunicarse con el sistema operativo mediante comandos escritos. En lugar de utilizar únicamente botones y menús, podemos escribir instrucciones para realizar diferentes tareas.

En el caso de Git, la terminal permite ejecutar comandos para crear repositorios, revisar archivos, preparar cambios, registrar commits y sincronizar el proyecto con GitHub. En Windows, una de las herramientas que se puede instalar junto con Git es Git Bash. Esta permite ejecutar comandos de Git mediante una interfaz de texto.

También existen otras opciones, como la terminal de macOS, la terminal de Linux y diferentes entornos de desarrollo que permiten trabajar con Git.

Por ejemplo, el comando git status permite consultar el estado del repositorio. De esta manera, podemos identificar si hay archivos nuevos, archivos modificados o cambios que todavía no se han registrado.

Es importante ejecutar los comandos en la carpeta correcta del proyecto y revisar sus resultados para asegurarnos de que las operaciones se realicen como esperamos.

1.8. COMANDOS BÁSICOS DE GIT

Git cuenta con diferentes comandos que permiten realizar las tareas más comunes durante el desarrollo de un proyecto. Cada comando cumple una función específica, por lo que es importante comprender para qué sirve antes de utilizarlo.

- **Comando git init**

El comando git init permite iniciar un nuevo repositorio Git en la carpeta donde nos encontramos trabajando. Se utiliza cuando queremos comenzar a controlar las versiones de un proyecto que todavía no tiene un repositorio Git.

Por ejemplo, si creamos una carpeta para organizar los documentos de una actividad, podemos iniciar un repositorio allí. A partir de ese momento, Git podrá comenzar a gestionar el historial de los cambios que registremos.

- **Comando git clone**

El comando git clone permite copiar un repositorio existente a nuestro equipo. Se utiliza, por ejemplo, cuando ya creamos un repositorio en GitHub y queremos tener una copia local para trabajar desde ella.

La estructura general del comando es:

git clone URL_DEL_REPOSITORIO

En este caso, se debe reemplazar **URL_DEL_REPOSITORIO** por la dirección real del repositorio que queremos copiar.

Cuando el proceso termina correctamente, se crea una carpeta con los archivos del repositorio y la información necesaria para trabajar con Git.

- **Comando git status**

El comando git status permite revisar el estado actual del repositorio. Este comando es útil porque muestra si hay archivos nuevos, modificaciones pendientes o cambios que ya están preparados para registrarse.

Por ejemplo, si agregamos un documento nuevo a la carpeta del proyecto, podemos utilizar git status para comprobar si Git lo identifica como un archivo sin seguimiento.

Consultar el estado del repositorio antes de registrar los cambios ayuda a evitar errores y permite saber qué archivos estamos a punto de incluir

- **Comando git add**

El comando git add permite preparar los cambios que queremos incluir en el siguiente commit.

Por ejemplo, si creamos un archivo llamado capitulo1.txt, podemos preparar ese archivo con el siguiente comando:

git add capitulo1.txt

Este comando indica que queremos incluir los cambios de ese archivo en el próximo commit.

Es importante comprender que git add no registra definitivamente los cambios en el historial. Su función es preparar los archivos para que posteriormente puedan guardarse mediante un commit.

- **Comando git commit**

El comando git commit permite registrar en el historial los cambios que previamente hemos preparado.

Cada commit debe tener un mensaje claro que explique qué se modificó. Esto facilita la revisión del trabajo y permite comprender cómo ha avanzado el proyecto.

Por ejemplo:

git commit -m "Agregar primer capítulo sobre Git"

El mensaje explica que se registró el primer capítulo relacionado con Git.

Es recomendable utilizar mensajes breves y descriptivos, en lugar de escribir frases generales como «cambios» o «actualización», porque estas no explican con claridad qué se hizo.

- **Comando git push**

El comando git push permite enviar al repositorio remoto los commits que todavía no se han enviado.

Por ejemplo, si trabajamos en nuestro repositorio local y registramos los cambios mediante un commit, podemos utilizar git push para enviarlos a GitHub.b

De esta manera, las modificaciones quedan disponibles en el repositorio remoto y otras personas con acceso pueden consultar la versión actualizada.

El comando exacto puede variar según la configuración del repositorio. Por eso, es importante revisar las instrucciones que Git muestra al configurar por primera vez la conexión con GitHub.

- **Comando git pull**

El comando git pull permite traer cambios del repositorio remoto e integrarlos en la rama local en la que estamos trabajando.

Es especialmente útil cuando se trabaja en equipo o cuando se han realizado modificaciones directamente en GitHub y necesitamos actualizar nuestra copia local.

Antes de continuar trabajando, conviene comprobar si existen cambios remotos que todavía no tenemos en el equipo.

¿Cómo se relacionan estos comandos?

Los comandos anteriores se pueden utilizar en diferentes momentos del trabajo. Por ejemplo, si tenemos un repositorio local y creamos un archivo nuevo, podemos revisar su estado, preparar el archivo, registrar los cambios y después enviarlos a GitHub.

El proceso general sería:

- gitstatus: revisar el estado de los archivos.
- git add capitulo1.txt: preparar el archivo que queremos registrar.
- git commit -m "Agregar primer capítulo": registrar los cambios en el historial.
- git push: enviar los commits al repositorio remoto.

Este proceso ayuda a mantener un registro ordenado de los avances. Sin embargo, cada comando tiene una función diferente y debe ejecutarse en el contexto adecuado.

1.9. IMPORTANCIA DE GIT Y GITHUB EN EL APRENDIZAJE

Aprender a utilizar Git y GitHub es importante porque estas herramientas permiten desarrollar hábitos de organización, control y seguimiento de los proyectos digitales.

En el ámbito académico, pueden utilizarse para guardar documentos, registrar las modificaciones realizadas y presentar evidencias del proceso de trabajo. Esto facilita que tanto el estudiante como el docente puedan consultar los archivos y revisar los avances.

En el ámbito laboral, Git es ampliamente utilizado en proyectos de programación y desarrollo de software. Permite que los equipos administren cambios, comparen versiones y colaboren de forma organizada.

GitHub, por su parte, facilita la publicación y el intercambio de proyectos. También permite que otras personas conozcan el trabajo realizado y participen en su desarrollo cuando cuentan con los permisos correspondientes. Además, aprender estas herramientas ayuda a comprender cómo se administran los proyectos digitales y por qué es importante mantener un historial de los cambios.

1.10. CONCLUSIÓN

Git y GitHub son herramientas que se complementan y permiten trabajar de manera más organizada con archivos y proyectos digitales. Git se encarga de controlar las versiones y registrar los cambios realizados, mientras que GitHub permite alojar los repositorios en línea y compartirlos con otras personas.

Durante el aprendizaje de estas herramientas, es importante comprender conceptos como repositorio, control de versiones, commit y sincronización. También es necesario aprender a utilizar comandos como git clone, git status, git add, git commit y git push.

REFERENCIAS BIBLIOGRÁFICAS

- **Chacon, S., y Straub, B. Pro Git.**